

HAUSARBEIT
Leonhard Lüdeke

Ein KI-Gamebot für Vier Gewinnt. Implementiert in Erlang

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Betreuung durch: Prof. Dr. Christoph Klause
Eingereicht am: 23. Januar 2026

HOCHSCHULE FÜR ANGEWANDTE
WISSENSCHAFTEN HAMBURG
Hamburg University of Applied Sciences

Inhaltsverzeichnis

1	Einleitung	2
1.1	Das Spiel Vier Gewinnt	2
2	Lösungsansätze	3
2.1	Minimax-Algorithmus	3
2.2	Alpha-Beta Optimierung	5
2.3	Monte-Carlo-Treesearch	6
2.4	Wahl des Algorithmus	7
3	Entwurf	8
3.1	Spielbrett	8
3.1.1	Datenstrukturen	9
3.1.2	Wichtigsten Funktionen	9
4	Realisierung des Entwurfs	11
5	Analyse	11
6	Fazit	11
	Literatur	11
	Selbstständigkeitserklärung	13

Diese Hausarbeit, die im Rahmen des Moduls Algorithmen und Datenstrukturen entstanden ist, beschäftigt sich mit dem Entwurf eines Gamebots für das Spiel Vier Gewinnt. Zu Beginn werden das Spiel und seine Regeln erklärt sowie bekannte Ansätze beleuchtet, die sich als Gamebot für Vier Gewinnt eignen. Anschließend wird ein Entwurfsmuster entwickelt, welches den ausgewählten Algorithmus (Gamebot) mit den gegebenen Codevorgaben (Das Spielbrett ist vorgegeben) darstellt. Darauf folgt die Beschreibung der Implementierung des Entwurfes, und eine Analyse des erstellten Codes.

Keywords: Vier Gewinnt, KI-Gamebot, Erlang

1 Einleitung

1.1 Das Spiel Vier Gewinnt

Vier Gewinnt ist ein klassisches Strategiespiel, und wurde 1973 von Howard Wexler handelt sich um ein endliches Zwei-Personen Nullsummenspiel mit perfekter Information. Ein Nullsummenspiel kennzeichnet sich dadurch aus, dass die Verluste des/der einen Spielers/in exakt den Gewinnen des/der anderen darstellen. Eine spielende Person ist deshalb perfekt informiert, da die gesamte Spielsituation jederzeit offen einsehbar ist. Somit kann ein/e Spieler/in eine voll informierte Entscheidung treffen. [6]

Das Spiel wird auf einem vertikal aufgerichteten Brett mit sieben Spalten und sechs Zeilen gespielt (7x6). Die Spielenden werfen abwechselnd Steine ihrer Farbe von oben in eine Spalte ein, wobei der Stein auf die unterste freie Position fällt. Dabei stehen jeder Person 21 Spielsteine zur Verfügung. Gewinner ist diejenige Person, die zuerst vier eigene Steine horizontal, vertikal oder diagonal in einer Linie anordnet. [8]

Die Kombination aus einfacher Spielmechanik und überraschend hohen Gesamtzahl der legalen Spielpositionen von circa 4,5 Billionen [2], während Tic-Tac-Toe 5.478 gültige Positionen besitzt, macht dieses Spiel zu einem interessanten Untersuchungsobjekt. Der Aufbau eines kompletten Spielbaumes ist, mit 4×10^{21} Blattknoten, aufwendig. Zusätzlich ist die Laufzeit, für die Berechnung des optimalen Spielzugs, in diesem Spielbaum praktisch unbrauchbar, wenn auch potenziell perfekt. Das Ermitteln des Perfekten Zuges, für einen Gamebot, liegt somit nicht in einer 1 zu 1 Abbildung aller Spielzustände, sondern in Algorithmen Ansätzen, welche ich im nächsten Unterkapitel beleuchtet werden.

An dieser Stelle sei erwähnt das Victor Allis [1] und zeitgleich auch James D. Allen [1] 1988 das Spiel vollständig lösten. Beide sind zu dem Ergebnis gekommen das die beginnende Person, bei perfektem Spielverhalten, immer gewinnt wenn sie den ersten Stein in die mittlere Spalte einwirft. Wenn nicht kommt endet das Spiel mit einem Unentschieden.

2 Lösungsansätze

Wie oben beschrieben ist es, aufgrund des großen Suchraumes, unpraktikabel alle Spielstände abzubilden. Der Unterschied, der einzelnen Lösungsansätze, liegt darin wie dieser Zustandsraum durchsucht und der Suchbaum aufgebaut wird. Alle Verfahren haben dabei gemein, dass sie den Zustandsraum mit Knoten (Stellungen im Spiel) und Kanten (ein Zug) darstellen. Nachfolgend sind Knoten und Kanten als diese Zustände zu verstehen. Je nachdem welcher Algorithmus beschrieben wird, können diese Zustände zusätzliche Eigenschaften bekommen.

2.1 Minimax-Algorithmus

Ein fundamentales Konzept zur Berechnung von optimalen Strategien in Nullsummenspielen ist der Minimax-Algorithmus [7]. Dieser Algorithmus basiert auf einem Spielbaum, in dem jeder Knoten ein MAX- oder MIN-Knoten ist. Diese geben, neben dem Spielzustand, an welcher Person (Maxi- oder Minimierer/in) am Zug ist und besitzen einen numerischen Wert. Dieser numerische Wert gibt an welcher Spielzustand besser für die jeweilige Person ist. Der sogenannte Maximierer/in (meist 1. Spieler/in) wählt immer den Spielzustand mit dem höheren Wert und der Minimierer/in (meist 2. Spieler/in), den kleineren Wert. Dies entspricht einer Optimalen Spielweise.

Die Werte haben ihren Ursprung in den Blattknoten (Endzustände) des Baumes, welche zeitgleich einen Terminalen Zustand, also das Ende eines Spiels darstellen. Diese besitzen eine heuristische Bewertungsfunktion, die aussagt wie gut oder schlecht dieser Endzustand für die Maximierende Person ist. Die Werte für die Inneren Knoten setzen sich aus Ihren Kindknoten zusammen, also durch das Propagieren der Werte Ihrer Kindknoten.. Der Wert an der Wurzel gibt somit den Wert des terminalen Zustands an, welcher beim Perfektem Spielverhalten beider Spieler erreicht wird. Die Abbildung 1 zeigt diese Variante und hebt den Pfad in rot hervor. Dieser Pfad wird Princibal-Variation genannt.

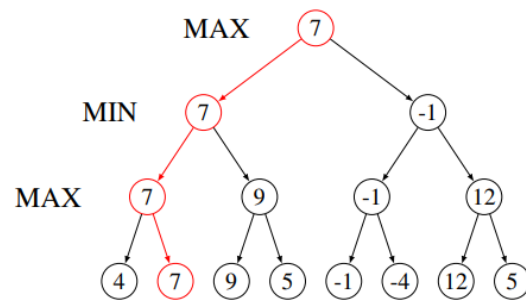


Abbildung 1: Abbildung eines Minimax-Spielbaumes [5]

Klassisch wird dieser Algorithmus mittels einer Tiefensuche implementiert und besitzt ohne Optimierung eine exponentielle Laufzeit von $O(b^d)$ wobei $b = \text{mögliche Spalten pro Zug}$ und $d = \text{Züge im Voraus}$ darstellen. Dabei werden zunächst die Minimax-Werte aller möglichen Nachfolgeknoten eines Knotens berechnet. Ist der aktuelle Knoten ein MAX-Knoten, übernimmt er den höchsten Minimax-Wert seiner Kindknoten. Handelt es sich hingegen um einen MIN-Knoten, erhält er den niedrigsten Minimax-Wert aus seinen Kindknoten. Somit lässt sich der Algorithmus aus zwei Teilen zusammensetzen.

Einer Funktion, die der minimierende Spieler verwendet, und einer, die vom maximierenden Spieler aufgerufen wird. Abhängig davon, ob der aktuell betrachtete Knoten ein MIN- oder MAX-Knoten ist, übernimmt dieser den Wert des Kindknotens mit dem jeweils kleinsten oder größten Wert. Dieses Vorgehen lässt sich jedoch zu einer einzigen Komponente vereinheitlichen, indem man das Minimierungsproblem des MIN-Spielers in ein Maximierungsproblem umwandelt. Dazu wird die Bewertungsfunktion des minimierenden Spielers einfach negiert, sodass beide Spieler effektiv ein Maximierungsproblem lösen. Diese vereinfachte Form ist als *Negamax-Verfahren* bekannt. Nachfolgend ein Beispiel in Pseudocode:

```
negamax(Knoten) :
    wenn Knoten ein Blatt ist:
        gib den Wert des Knotens zurück
    Wert = -minimalerNegativWert
    für alle Kinder von Knoten:
        Wert = max(Wert, -negamax(Kind))
    gib Wert zurück
```

2.2 Alpha-Beta Optimierung

Die Alpha-Beta-Pruning-Technik ist eine Optimierungsmethode des Minimax-Algorithmus, die durch intelligentes Abschneiden von Spielbaumästen die Anzahl der zu evaluierenden Knoten erheblich reduziert [3]. Das Verfahren arbeitet mit zwei Werten:

- **Alpha** (α): Die beste (höchste) Bewertung, die der Maximierer bisher gefunden hat.
- **Beta** (β): Die beste (niedrigste) Bewertung, die der Minimierer bisher gefunden hat.

Dabei stellte *Knuth* [3] in seiner Analyse eine obere und untere Wertschranke dar welche folgendes Suchfenster ergibt:

$$\alpha \leq \text{MinimaxWert} \leq \beta$$

Ein Ast kann demnach abgeschnitten werden, wenn ein besserer Minimax-Wert außerhalb des Suchfensters gefunden wird, da die restlichen Züge in diesem Ast nicht mehr relevant für die optimale Spielentscheidung sind. Dies basiert auf der Annahme, dass beide Spieler optimal spielen und keine schlechteren Positionen akzeptieren würden [3].

In der Regel werden α und β auf den schlechtesten Wert für den jeweiligen Spieler gesetzt. Für $\alpha = -\infty$ und für $\beta = \infty$. Diese werden aktualisiert und die jeweiligen Knoten aktualisiert und auf die Bedingung des Suchfensters geprüft. In der Abbildung 2 ist zu sehen wie sich der Suchbaum reduziert. Um nicht zwei verschiedene Algorithmen implementieren zu müssen kann die vorhin beschriebene *Negamax* variante angewandt werden. Hier ein Beispiel als Pseudocode:

```
AlphaBetaNegamax(Knoten, Alpha, Beta):  
    wenn der Knoten ein Blatt ist:  
        gib den Wert des Knotens zurück  
    maxWert = -maximalNegativWert  
    für jeden Nachfolger des Knotens:  
        Wert = -AlphaBetaNegamax(Nachfolger, -Beta, -Alpha)  
        maxWert = max(maxWert, Wert)  
    Alpha = max(Alpha, Wert)  
    wenn Alpha >= Beta:  
        breche ab  
    gib maxWert zurück
```

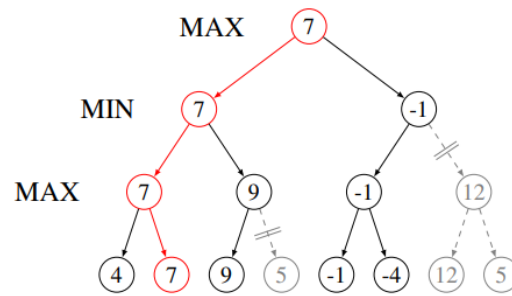


Abbildung 2: Abbildung eines Minimax-Spielbaumes [5]

Die Effektivität von Alpha-Beta-Pruning hängt stark von der Reihenfolge ab, in der die Züge evaluiert werden. Mit optimaler Zugreihenfolge ist eine Reduktion der Komplexität auf $O(b^{d/2})$ möglich, was eine quadratische Verbesserung gegenüber dem klassischen Minimax darstellt [3].

2.3 Monte-Carlo-Treesearch

Ein alternatives Konzept zur Berechnung von annähernd optimalen Strategien in Nullsummenspielen ist die Monte-Carlo-Baumsuche (MCTS). Wie der Minimax-Algorithmus basiert auch MCTS auf einem Spielbaum, in dem jeder Knoten einen Spielzustand repräsentiert und einen numerischen Wert besitzt. Im Gegensatz zum Minimax-Algorithmus, der durch statische Bewertungsfunktionen an den Blattknoten operiert, bestimmt MCTS diese Werte durch wiederholte stochastische Simulationen (Rollouts) bis zu den Endzuständen des Spielbaums. Durch die Propagierung der Simulationsergebnisse von den Blattknoten zu inneren Knoten konvergiert die Schätzung der Knotenwerte mit hinreichend vielen Iterationen gegen gute Näherungen der optimalen Spielqualität [4].

Die MCTS-Methode durchläuft zur Wertberechnung vier konzeptionelle Phasen: In der Selection-Phase werden bestehende Knoten gemäß der UCB-Heuristik ausgewählt, die Exploration und Exploitation balanciert. Die Expansion-Phase fügt neue Knoten hinzu, während die Simulation-Phase Zufallsspielzüge bis zum Endzustand durchführt. In der abschließenden Backpropagation-Phase werden die Ergebnisse dieser Simulationen entlang des durchlaufenen Pfades rückwärts propagiert und aktualisieren die Knotenwerte sowie Besuchszahlen.

Diese Methode wird iterativ implementiert und benötigt, im Gegensatz zum klassischen Minimax mit Tiefensuche, keine explizite Bewertungsfunktion. Während Minimax ohne

Optimierung exponentielle Laufzeit von $O(b^d)$ aufweist, wobei $b = \text{mögliche Spalten pro Zug}$ und $d = \text{Züge im Voraus}$, funktioniert MCTS praktikabler bei Spielen mit moderater Komplexität wie 4 Gewinnt. Mit nur 32 MB Speicher und etwa 500.000 Simulationen erreicht MCTS zuverlässige Spielstärke. Damit bietet MCTS eine effiziente Alternative zu Alpha-Beta Pruning, insbesondere wenn keine domänenspezifische Evaluierungsfunktion vorhanden ist.

2.4 Wahl des Algorithmus

Für diese Implementierung habe ich mich für den **Minimax-Algorithmus mit Alpha-Beta-Pruning** entschieden [7, 3]. Diese Wahl wird durch folgende Faktoren begründet:

1. **Vorhersagbarkeit und Optimalität:** Minimax mit Alpha-Beta-Pruning garantiert die Findung des optimalen Zuges unter Annahme optimalen gegnerischen Spiels, sofern die Suchtiefe ausreichend ist [7].
2. **Deterministische Spielweise:** Das Verfahren liefert reproduzierbare Ergebnisse, was für Analysen und Tests vorteilhaft ist.
3. **Effiziente Komplexitätsreduktion:** Mit Alpha-Beta-Pruning kann die Komplexität quadratisch reduziert werden, was tiefere Suchbäume ermöglicht. [3].
4. **Domänenspezifische Optimierung:** Für Vier Gewinnt kann eine spezialisierte Bewertungsfunktion entwickelt werden, die strategische Ziele quantifiziert [7].

Zuguter Letzt viel die Wahl ebenfalls auf eine Implementierung mittels des Minimax Algorithmuses, da ich den MCTS schon einmal Implementiert habe.

3 Entwurf

3.1 Spielbrett

Das Spielbrett ist Zentral für das Spiel 4 Gewinnt und wurde vom Professor Dr. Klauck vorgegeben. Die Schnittstellendefinition sieht folgende Operatoren vor:

- new_board/0: $\rightarrow$ board
- is_board/1: board $\rightarrow$ boolean
- export_board/1: board $\rightarrow$ boardMatrix
- print_board/1: board | boardMatrix $\rightarrow$ done
- next_player/1: board | {<blue | red>, <count>} $\rightarrow$ {<red | blue>, <count>+1}
- current_player/1: board $\rightarrow$ {<red | blue>, <count>}
- score/1: board $\rightarrow$ <running | red | blue | remis>
- possible_moves/1: board $\rightarrow$ number-list
- possible_moves_pos/1: board $\rightarrow$ number-list
- make_move/2: board $\times$ number $\rightarrow$ board | {error, {game_over, <score>}}
| {error, column_full}
| {error, column_outofrange}
- score_ChipsRow/1: board $\rightarrow$ number-list
- score_ChipsRow/2: board $\times$ {<red | blue>, <count>} $\rightarrow$ number-list
- check_geq_three/2: board $\times$ {<red | blue>, <count>} $\rightarrow$ number-list

Auf Basis dieser Operatoren habe ich die Datenstrukturen und zusätzliche Funktionen überlegt die bei der Schnittstellen Realisierung unterstützen. Einfache Getter und potenzielle Hilfsfunktionen sind hier nicht aufgelistet.

3.1.1 Datenstrukturen

Board (Hauptdatenstruktur)

`Board = {board, Matrix, CurrentPlayer, Score}`

- `Matrix`: Liste von 7 Spalten (Spaltenweise Organisation aufgrund der Spielmechanik.)
- `CurrentPlayer`: `{Color, Count}` mit `Color` $\in$ `{red, blue}`, `Count` ≥ 0
- `Score`: `running | red | blue | remis`

Spalte

`Column = [Cell1, Cell2, ..., Cell6]`

- Liste von 6 Zellen (von unten nach oben)
- `Cell`: `empty | {Color, Count}`

Koordinatensystem

- Links unten: (1,1), Rechts oben: (7,6)
- Spalten: 1–7 (horizontal), Reihen: 1–6 (vertikal)

3.1.2 Wichtigsten Funktionen

`new_board/0` $\rightarrow$ `Board` [E1]

Initialisiert das Spiel und bedarf daher einer Näheren Betrachtung.

- Erzeuge 7 leere Spalten mit je 6 Zellen (`empty`)
- Setze `{red, 0}` als Startspieler, `Score = running`

make_move/2 (Board, ColIdx) → Board | Error [E2]

Stellt die Kernfunktionalität dar, führt Spielzüge aus und aktualisiert den Zustand.

1. Prüfe: `Score = running? → sonst {error, {game_over, Score}}`
2. Prüfe: `1 ≤ ColIdx ≤ 7 → sonst {error, column_outofrange}`
3. Finde erste leere Zelle in Spalte `ColIdx` (von unten)
 - Falls keine: `{error, column_full}`
4. Setze Chip `{CurrentPlayer}` in gefundene Zelle
5. Berechne neuen Score via `score/1`
6. Berechne `NextPlayer = {andere Farbe, Count+1}`
7. Gebe `{board, NewMatrix, NextPlayer, NewScore}` zurück

score/1 (Board) → Status [E3]

Diese Funktion berechnet welcher Spieler Gewonnen bzw. verloren hat und ist damit essentiell.

1. Prüfe `winner_check(Matrix, red) → true: red`
2. Sonst prüfe `winner_check(Matrix, blue) → true: blue`
3. Sonst prüfe `is_full(Matrix) → true: remis`
4. Sonst: `running`

winner_check(Matrix, Color) → Boolean [E3.1]

Eine sehr wichtige Hilfsfunktion für `score/1` und prüft ob 4 Steine nebeneinander liegen in allen Richtungen:

- Horizontal: Iteriere über Reihen
- Vertikal: Iteriere über Spalten
- Diagonal $\searrow$: Alle Diagonalen von links-oben nach rechts-unten
- Diagonal $\nearrow$: Alle Diagonalen von links-unten nach rechts-oben

score_ChipsRow/2 (Board, Player) → NumberList [E4]

Wie in der Schnittstelle beschrieben, wird hier berechnet, wie viele gleichfarbige Chips um das nächste Freie Feld herum liegen. Die Vorgehensweise:

Für jede Spalte i (1..7):

1. Falls Spalte voll: -1 (Somit ist die nicht Spielbar)
2. Sonst: Bestimme Position (ReihenIndex, i) des nächsten freien Feldes
3. Zähle in alle 8 Richtungen gleichfarbige Chips:
 - Links, Rechts, Oben, Unten, 4 Diagonalrichtungen
4. Summiere alle Nachbarschaften $\rightarrow [\text{Count}_1, \dots, \text{Count}_7]$

check_geq_three/2 (Board, Player) → NumberList [E5]

Taktische Analyse – identifiziert kritische Spalten (Gewinn/Block)

- Nutze `score_ChipsRow/2` zur Berechnung
- Filter Spalten mit $\text{Count} \geq 3 \rightarrow [\text{ColIdx}_1, \text{ColIdx}_2, \dots]$

4 Realisierung des Entwurfs

% !TEX root = ../termpaper.tex % @author Leonhard Lüdeke %

5 Analyse

6 Fazit

Literatur

- [1] ALLEN, James D.: Expert play in connect-four. In: *Verfügbar unter <https://web.archive.org/web/20131023004851/http://homepages.cwi.nl/~tromp/c4.html>* Allis 1988 (1990)
- [2] DENGEL, Andreas ; BERNS, Karsten ; BREUEL, Thomas M. ; BOMARIUS, Frank ; ROTH-BERGHOFER, Thomas R.: *KI 2008: Advances in Artificial Intelligence: 31st Annual German Conference on AI, KI 2008, Kaiserslautern, Germany, September 23-26, 2008, Proceedings*. Bd. 5243. Springer Science & Business Media, 2008
- [3] KNUTH, Donald E. ; MOORE, Ronald W.: An analysis of alpha-beta pruning. In: *Artificial Intelligence* 6 (1975), Nr. 4, S. 293–326. – URL <https://www.sciencedirect.com/science/article/pii/0004370275900193>. – ISSN 0004-3702
- [4] KOCSIS, Levente ; SZEPESVÁRI, Csaba: Bandit based Monte-Carlo Planning. In: *Proceedings of the European Conference on Machine Learning (ECML)*, Springer, 2006, S. 282–293
- [5] LIESE, Yassin: *Systematische Evaluierung und Optimierung Minimax-basierter Suchstrategien anhand des Spiels Vier-Gewinnt*. Februar 2024. – URL <https://doc.neuro.tu-berlin.de/bachelor/2024-BA-YassinLiese.pdf>
- [6] OWEN, Guillermo: *Spieltheorie*. Springer-Verlag, 2013
- [7] RUSSELL, Stuart J. 1. ; NORVIG, Peter 1. (Hrsg.): *Artificial intelligence a modern approach*. 3. ed., international ed. Boston Munich u.a. : Pearson, 2010 (Prentice-Hall series in artificial intelligence). – URL <http://www.gbv.de/dms/tib-ub-hannover/627596169.pdf>. – Literaturverz. S. 1063 - 1093

- [8] WIKIPEDIA: *Vier gewinnt* — *Wikipedia, The Free Encyclopedia*. <http://de.wikipedia.org/w/index.php?title=Vier%20gewinnt&oldid=260829991>. 2025. – [Online; accessed 26-November-2025]

Erklärung zur selbständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Die in meiner Arbeit verwendeten KI-basierten Hilfsmittel habe ich (ggf. mit Produktnamen) angegeben.

Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Mir ist bewusst, dass wahrheitswidrige Angaben als Täuschungsversuch behandelt werden können.

Ort

Datum

Unterschrift im Original